Desarrollar software a partir de la integración de sus módulos componentes

Mario Hernando Gallo Gallego

GAES # 6

Servicio Nacional de Aprendizaje – (SENA)

Análisis y Desarrollo de Software – (3118493)

Andrés Rubiano Cucarían

Cali, Colombia

22 de marzo de 2026

I. Introducción

El presente proyecto tiene como propósito el desarrollo de un módulo funcional dentro del sistema de gestión de la empresa Proterquim, enfocado en la administración de empleados. Este módulo forma parte de un sistema más amplio orientado al control de inventarios y gestión de productos químicos.

El desarrollo se llevó a cabo utilizando tecnologías modernas de desarrollo web, implementando una arquitectura cliente-servidor que permite la comunicación entre el frontend y el backend mediante servicios REST. Se empleó Angular para la construcción de la interfaz de usuario y Node.js con Express para la lógica del servidor, junto con una base de datos MongoDB para el almacenamiento de la información.

Este proyecto permitió aplicar conocimientos relacionados con análisis, diseño, desarrollo y pruebas de software, así como la implementación de buenas prácticas de programación y estructuración de sistemas.

II. 2. Objetivo General

Desarrollar un módulo web para la gestión de empleados de la empresa Proterquim, que permita realizar operaciones de registro, consulta, actualización y eliminación de información, aplicando buenas prácticas de desarrollo de software.

Objetivos Específicos

- ✓ Diseñar la estructura del sistema utilizando arquitectura por capas.
- ✓ Implementar un CRUD completo para la gestión de empleados.
- ✓ Desarrollar una interfaz de usuario amigable utilizando Angular.
- ✓ Crear un backend funcional mediante Node.js y Express.
- ✓ Integrar el frontend con el backend a través de servicios HTTP.
- ✓ Aplicar buenas prácticas de codificación y organización del código.
- ✓ Realizar pruebas funcionales para verificar el correcto funcionamiento del sistema.

III. Alcance del Proyecto

El proyecto abarca el desarrollo del módulo de gestión de empleados dentro del sistema de Proterquim, incluyendo:

- ✓ Registro de empleados
- ✓ Visualización de datos en una tabla

- ✓ Edición de información
- ✓ Eliminación de registros
- ✓ Filtrado de empleados

No incluye:

- ✓ Autenticación de usuarios
- ✓ Roles y permisos
- ✓ Integración completa con inventarios de productos químicos

IV. Desarrollo del Proyecto

5.1 Requerimientos del Sistema

Requerimientos Funcionales

Los requerimientos funcionales definen las acciones que el sistema debe realizar. Para el desarrollo del módulo de gestión de empleados se tuvieron en cuenta los siguientes:

- ❖ **RF01 – Registrar empleados**

El sistema debe permitir registrar nuevos empleados ingresando nombre, cargo y salario.

- ❖ **RF02 – Listar empleados**

El sistema debe mostrar en una tabla todos los empleados registrados en la base de datos.

- **RF03 – Buscar empleados**

El sistema debe permitir filtrar empleados por nombre o cargo mediante un campo de búsqueda.

- **RF04 – Editar empleados**

El sistema debe permitir modificar la información de un empleado existente.

- **RF05 – Eliminar empleados**

El sistema debe permitir eliminar empleados del sistema con confirmación previa.

- **RF06 – Mostrar mensajes del sistema**

El sistema debe mostrar mensajes de éxito o error al realizar operaciones (crear, editar, eliminar).

- **RF07 – Validación de campos**

El sistema debe validar que todos los campos obligatorios estén diligenciados antes de registrar o actualizar un empleado.

Requerimientos No Funcionales

Los requerimientos no funcionales describen cómo debe comportarse el sistema.

- **RNF01 – Usabilidad**

El sistema debe contar con una interfaz clara, intuitiva y fácil de usar para el usuario.

RNF02 – Rendimiento

El sistema debe responder de manera rápida a las solicitudes del usuario (listar, crear, editar, eliminar).

- **RNF03 – Seguridad**

El sistema debe validar la información enviada al backend y evitar operaciones inválidas (por ejemplo, eliminar sin ID).

- **RNF04 – Escalabilidad**

El sistema debe permitir la integración de nuevos módulos como productos, pedidos e inventarios.

- **RNF05 – Mantenibilidad**

El código debe estar organizado en componentes y servicios para facilitar futuras modificaciones.

- **RNF06 – Compatibilidad**

El sistema debe ejecutarse correctamente en navegadores web modernos.

- **RNF07 – Arquitectura**

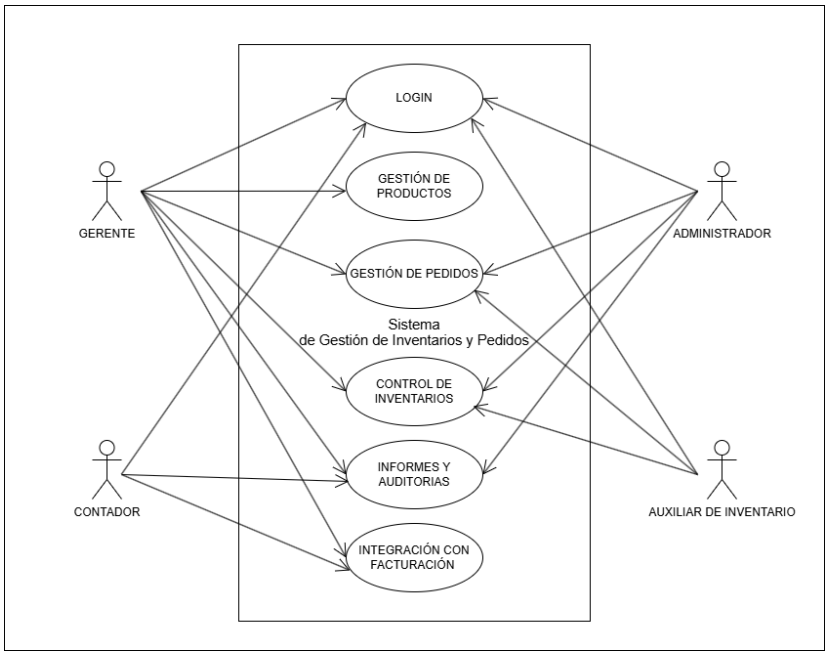
El sistema debe seguir una arquitectura cliente-servidor, separando frontend (Angular) y backend (Node.js).

5.2 Diagramas del Sistema

El sistema desarrollado se basa en una arquitectura previamente definida mediante diferentes diagramas UML, los cuales permiten comprender la estructura y funcionamiento general de la aplicación.

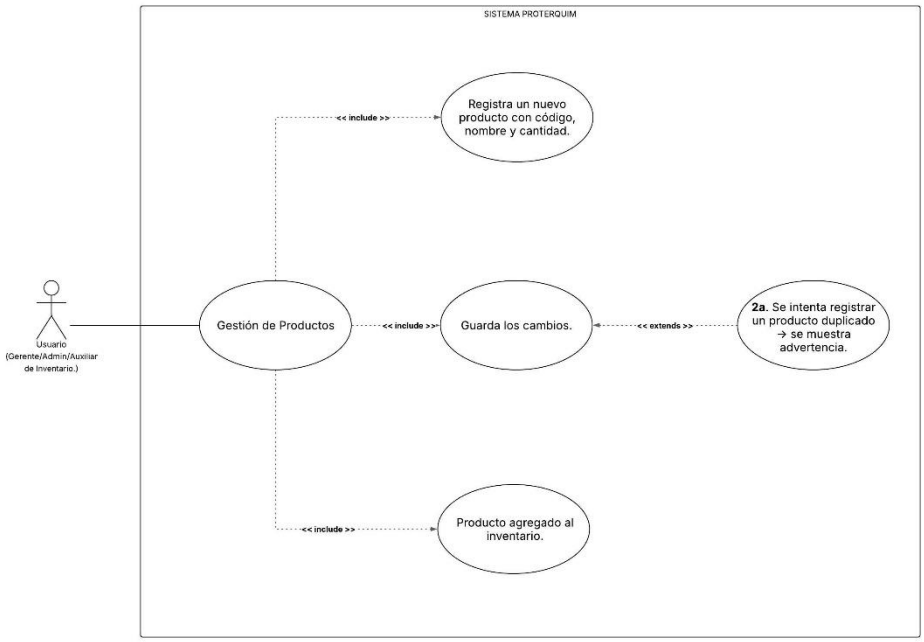
Diagrama de Casos de Uso Global

El sistema cuenta con un diagrama de casos de uso general que representa la interacción de los diferentes actores con los módulos principales del sistema.



Caso de Uso Específico (Gestión de Productos)

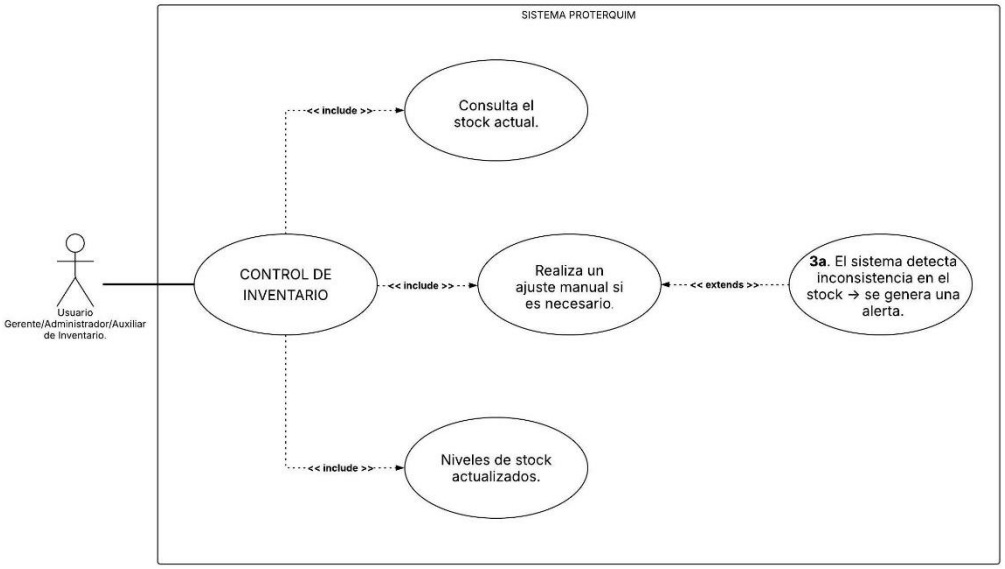
Como complemento, se presenta un caso de uso detallado del módulo de gestión de productos, el cual permite evidenciar el flujo de registro, validación y almacenamiento de información dentro del sistema.



Caso de Uso Específico (Control de Inventarios)

Como parte del sistema general, se presenta el caso de uso correspondiente al módulo de control de inventarios, el cual permite consultar el stock disponible, realizar ajustes manuales y detectar inconsistencias en los niveles de inventario.

Este módulo garantiza la integridad de los datos y la correcta gestión de los productos dentro del sistema.



Aunque el presente desarrollo se enfoca en el módulo de gestión de empleados, este hace parte de un sistema integral que incluye los casos de uso del sistema.

Diagrama de Clases

El sistema está estructurado mediante diferentes entidades como usuarios, productos, pedidos, facturas y stock, las cuales se relacionan entre sí para garantizar la correcta gestión de la información.

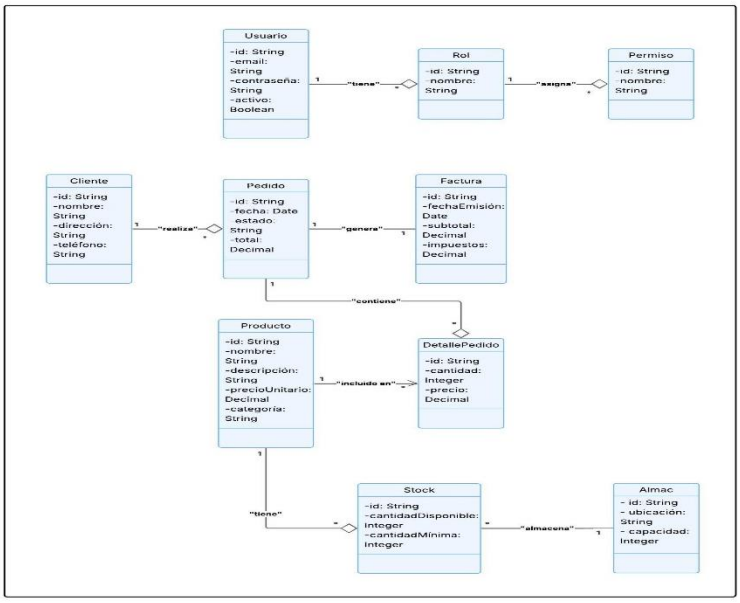


Diagrama de Paquetes

El sistema está organizado en paquetes que agrupan las diferentes funcionalidades, tales como inventario, pedidos, facturación, usuarios y seguridad.

Dentro del paquete de usuario se contempla la gestión de empleados, lo cual se relaciona directamente con el módulo desarrollado en la presente evidencia.

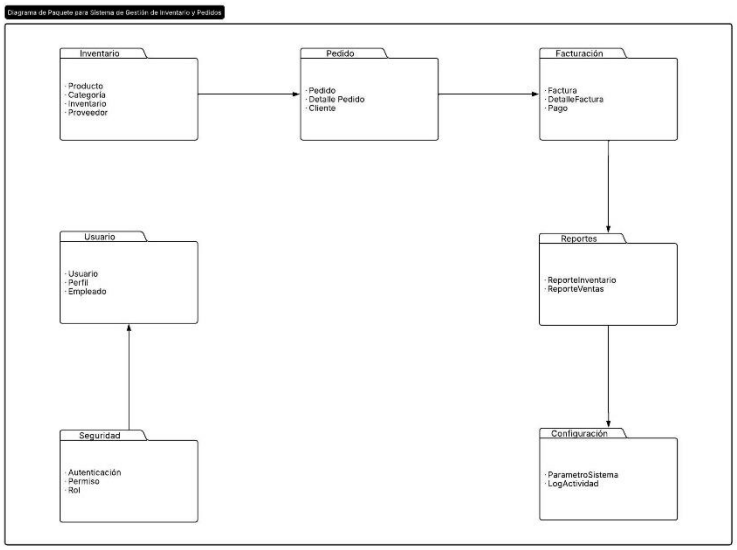
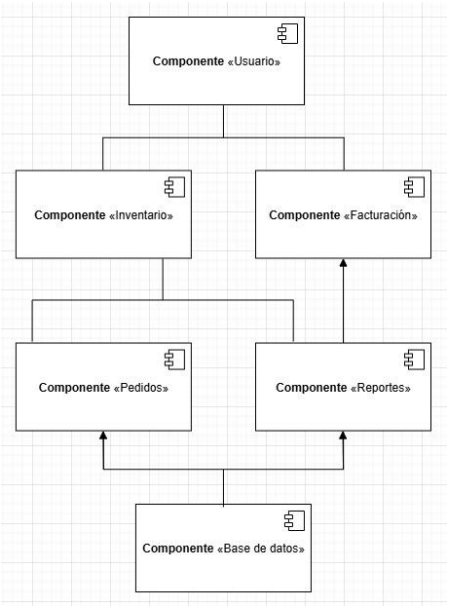


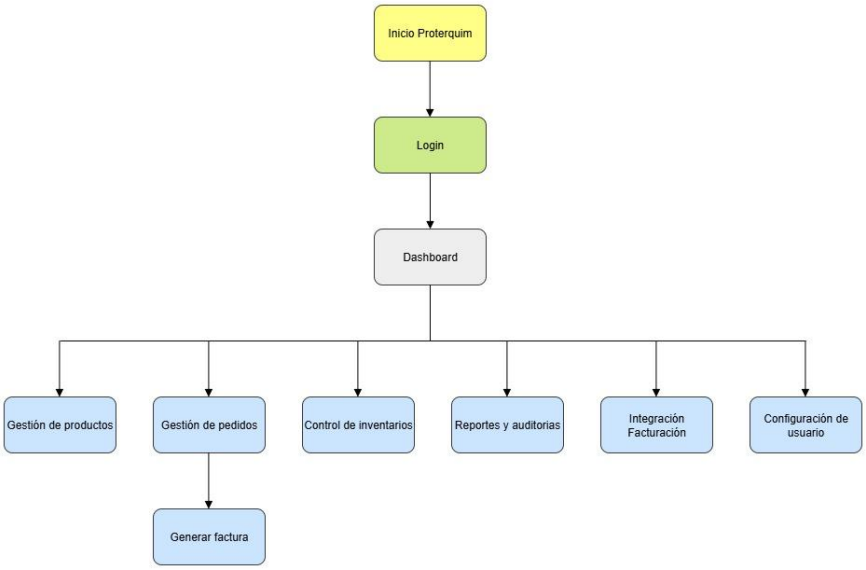
Diagrama de Componentes

El sistema se encuentra estructurado en diferentes componentes que permiten la organización de las funcionalidades principales, tales como usuario, inventario, pedidos, facturación y reportes, los cuales interactúan con una base de datos central.



Mapa de Navegación

El sistema cuenta con un flujo de navegación estructurado que inicia desde el acceso al sistema mediante autenticación, seguido del acceso al panel principal o dashboard, desde donde el usuario puede gestionar los diferentes módulos del sistema.



5.3 Metodología de Desarrollo

El desarrollo del sistema se realizó bajo un enfoque ágil incremental, el cual consiste en construir el software en diferentes etapas o módulos funcionales.

Cada incremento del sistema permite añadir nuevas funcionalidades, como la gestión de empleados, productos, pedidos e inventarios, garantizando que el sistema evolucione de manera progresiva y controlada.

Este enfoque facilita la identificación de errores, la mejora continua del sistema y la adaptación a nuevos requerimientos en donde el sistema fue construido en etapas:

- ✓ Diseño de la interfaz
- ✓ Creación del backend
- ✓ Integración de servicios
- ✓ Pruebas funcionales

5.4 Arquitectura del Sistema

Cada componente del sistema fue ubicado dentro de una capa específica, garantizando la separación de responsabilidades y facilitando el mantenimiento del sistema. El sistema fue desarrollado bajo una arquitectura de tres capas:

- **Capa de Presentación**

Encargada de la interacción con el usuario, desarrollada en Angular.

- **Capa de Lógica de Negocio**

Implementada con Node.js y Express, donde se gestionan las reglas del sistema.

- **Capa de Datos**

Base de datos MongoDB encargada del almacenamiento de la información.

5.5 Implementación del CRUD

El sistema implementa operaciones CRUD, fundamentales en el desarrollo de aplicaciones:

- **CREATE (Crear)**

Permite registrar nuevos empleados mediante un formulario.

- **READ (Leer)**

Consulta y visualiza la lista de empleados almacenados.

- **UPDATE (Actualizar)**

Permite modificar la información de un empleado existente.

- **DELETE (Eliminar)**

Permite eliminar un empleado del sistema.

5.6 Métodos Implementados

En el frontend se utilizaron métodos como:

`listarEmpleados()`: Obtiene los datos desde el backend.

`crearEmpleado()`: Envía datos para registrar un nuevo empleado.

`editarEmpleado()`: Permite seleccionar un empleado para edición.

`actualizarEmpleado()`: Guarda los cambios realizados.

`eliminarEmpleado()`: Elimina un registro existente.

En el backend:

```
GET /api/empleados
```

```
POST /api/empleados
```

```
PUT /api/empleados/:id
```

```
DELETE /api/empleados/:id
```

5.7 Tecnologías Utilizadas

Frontend:

- ✓ Angular
- ✓ TypeScript
- ✓ HTML5
- ✓ CSS3

Backend:

- ✓ Node.js
- ✓ Express

Base de Datos:

- ✓ MongoDB

Herramientas:

- ✓ Visual Studio Code
- ✓ Git

5.8 Organización del Código

El proyecto se estructuró en componentes reutilizables:

- ✓ Componentes: interfaz y lógica visual
- ✓ Servicios: comunicación con el backend
- ✓ Modelos: estructura de datos

En el backend se estructuró el proyecto en carpetas como:

- ✓ routes: definición de endpoints
- ✓ controllers: lógica de negocio
- ✓ models: estructura de datos en MongoDB
- ✓ config: conexión a la base de datos

5.9 Buenas Prácticas Aplicadas

- ✓ Uso de interfaces en TypeScript
- ✓ Separación de responsabilidades
- ✓ Nombres claros en variables y funciones
- ✓ Código modular y reutilizable
- ✓ Validación de datos

5.10 Seguridad

Se implementaron mecanismos básicos como:

- ✓ Validación de campos obligatorios
- ✓ Control de datos enviados al servidor
- ✓ Manejo de errores en peticiones HTTP
- ✓ Uso de CORS para permitir la comunicación entre frontend y backend.
- ✓ Manejo de peticiones HTTP seguras.

5.11 Pruebas Realizadas

Se realizaron pruebas funcionales con el fin de verificar el correcto funcionamiento del sistema, incluyendo las operaciones como:

- ✓ Creación de empleados
- ✓ Visualización de datos

- ✓ Edición correcta
- ✓ Eliminación de registros

5.12 Configuración del Entorno

- ✓ Backend: Puerto 3000
- ✓ Frontend: Puerto 4200
- ✓ Base de datos: MongoDB local

5.13 Pruebas del Sistema

Las pruebas del sistema se realizaron mediante la verificación de cada una de las operaciones CRUD implementadas en el backend y su correcta visualización en el frontend.

A continuación, se presentan las pruebas realizadas:

Prueba 1: Crear Empleado (CREATE)

Método: POST

URL: http://localhost:3000/api/empleados

Body:

{

"nombre": "Carlos López",

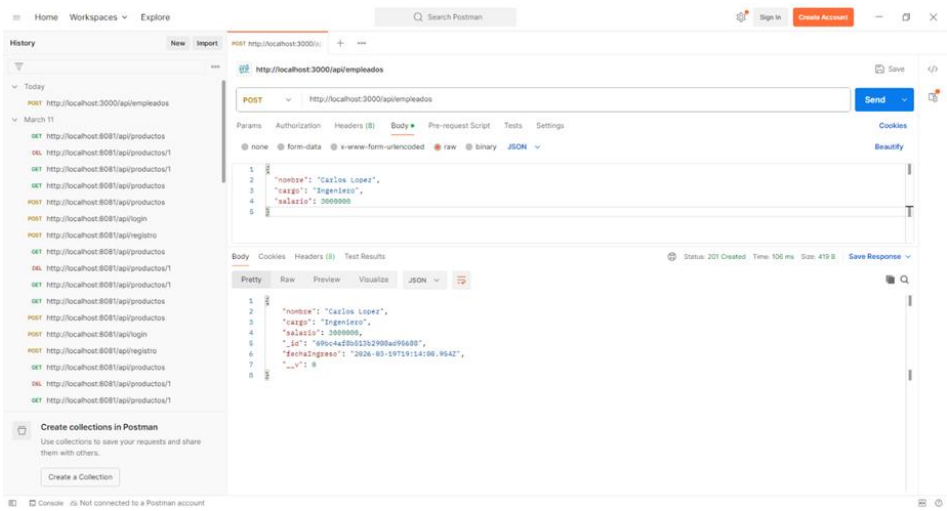
"cargo" : "Ingeniero",

"salario": 3000000

}

Resultado:

El empleado fue registrado correctamente en la base de datos y visualizado en la interfaz.



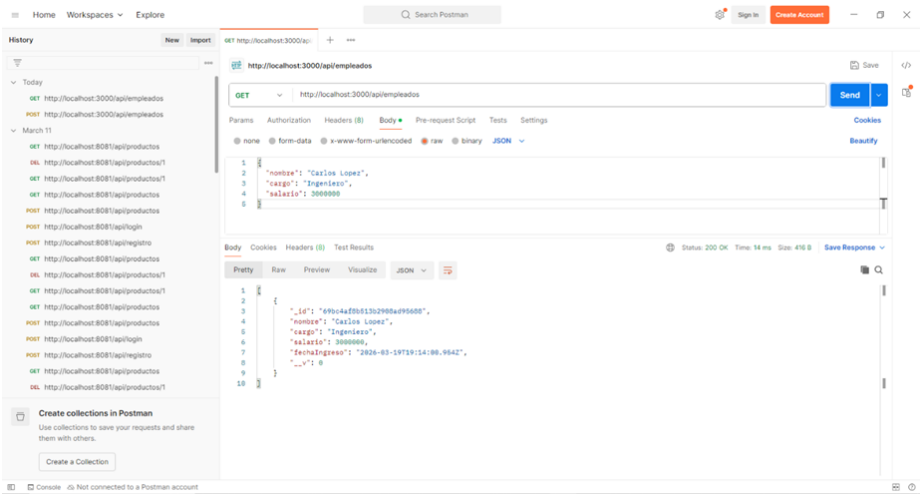
Prueba 2: Listar Empleados (READ)

Método: GET

URL: http://localhost:3000/api/empleados

Resultado:

Se obtuvo correctamente el listado de empleados desde el backend y se visualizó en la tabla del sistema.



Prueba 3: Actualizar Empleado (UPDATE)

Método: PUT

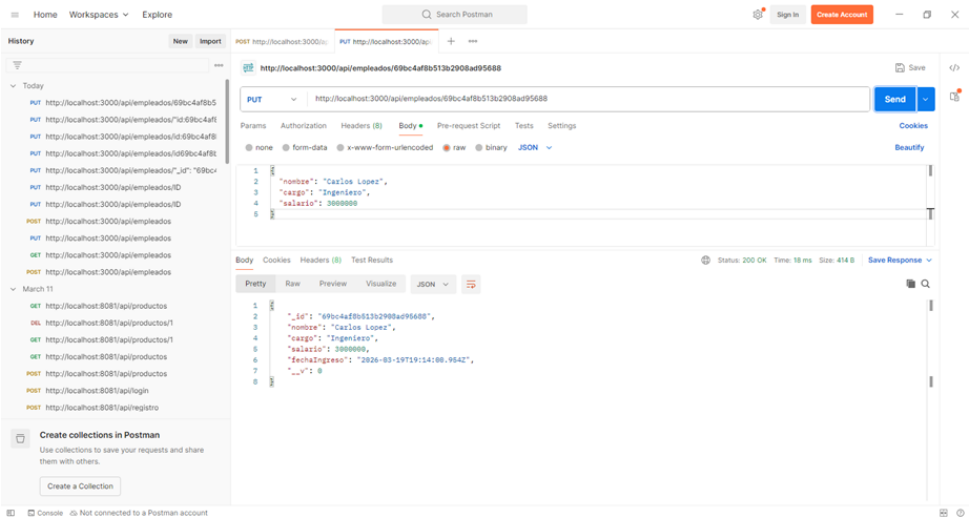
URL: http://localhost:3000/api/empleados/{id}

Body:

```
{
  "nombre": "Carlos Modificado",
  "cargo": "Ingeniero",
  "salario": 3500000
}
```

Resultado:

El empleado fue actualizado correctamente y los cambios se reflejaron en la interfaz.



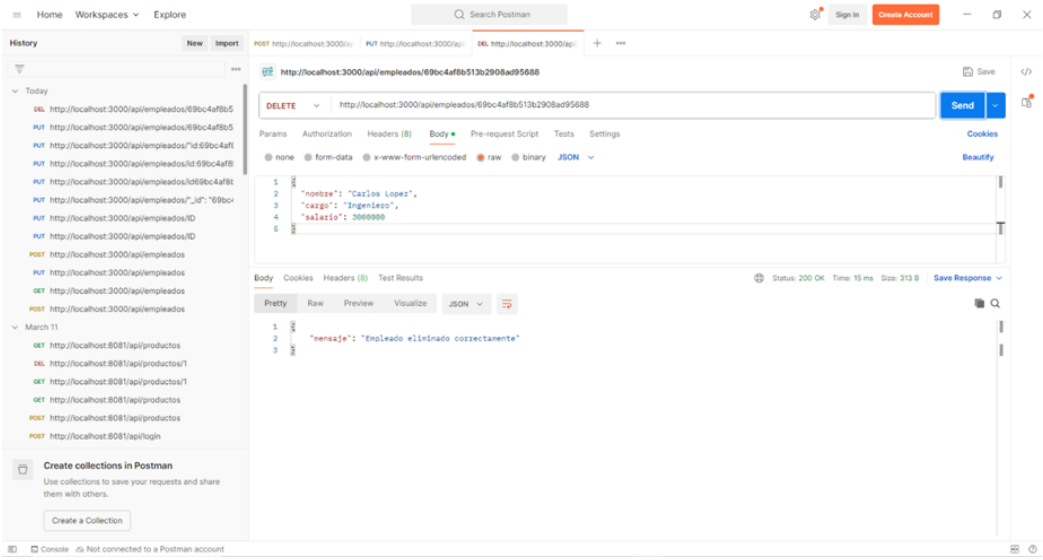
Prueba 4: Eliminar Empleado (DELETE)

Método: DELETE

URL: http://localhost:3000/api/empleados/{id}

Resultado:

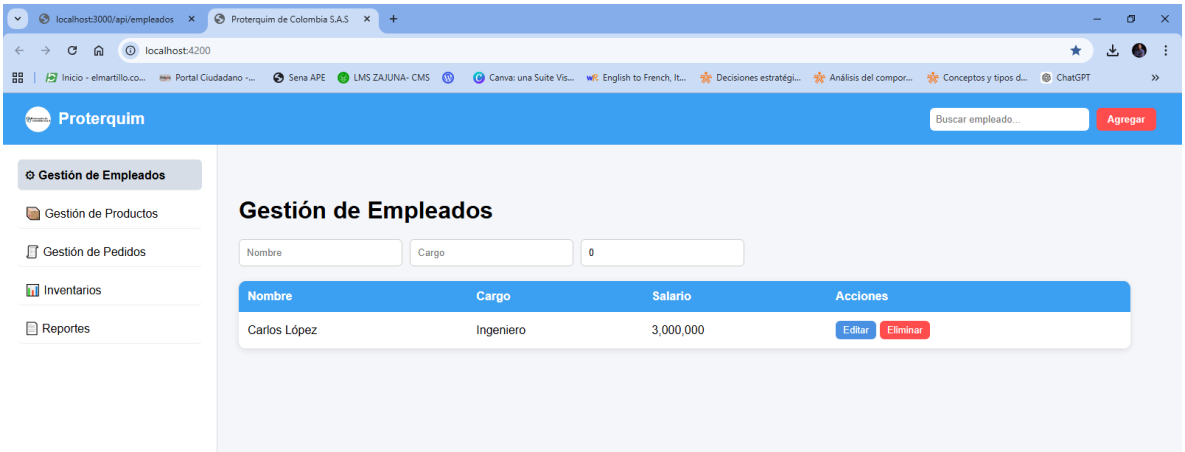
El empleado fue eliminado correctamente de la base de datos y dejó de mostrarse en la tabla.



Interfaz del Sistema (Frontend)

La siguiente imagen presenta la interfaz del módulo de gestión de empleados desarrollada en Angular. En esta se puede observar el formulario de registro, el campo de búsqueda para filtrar empleados y la tabla donde se visualizan los datos almacenados en la base de datos.

Asimismo, se evidencian las acciones disponibles como editar y eliminar registros, lo cual permite la gestión completa de la información mediante operaciones CRUD. La interfaz fue diseñada con el objetivo de ser intuitiva, clara y fácil de usar para el usuario.



Ejecución del Backend

La siguiente imagen muestra la ejecución del servidor backend desarrollado con Node.js y Express desde el entorno de desarrollo Visual Studio Code. En la terminal se evidencia el inicio correcto del servidor en el puerto 3000, lo que permite la disponibilidad de los servicios REST para la gestión de empleados.

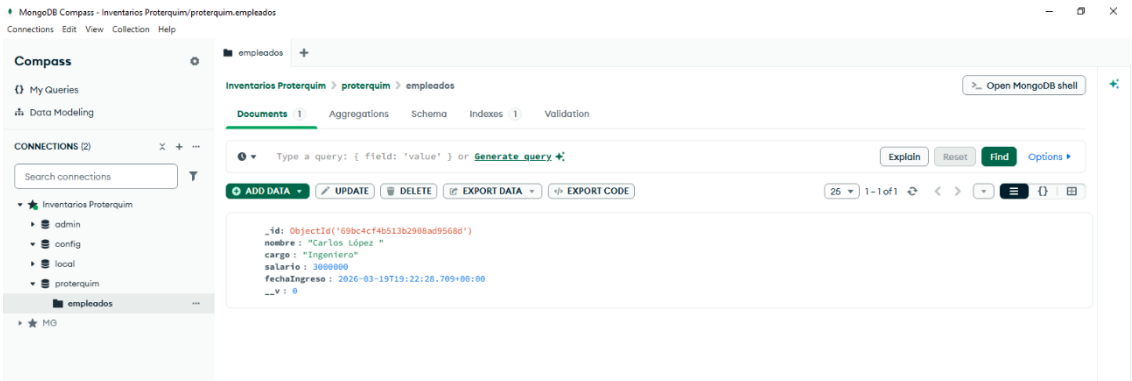
A través de este servidor se manejan las operaciones CRUD, permitiendo la comunicación con la base de datos MongoDB y la integración con el frontend desarrollado en Angular.

Base de Datos (MongoDB)

La siguiente imagen muestra la base de datos MongoDB utilizada en el desarrollo del sistema, visualizada mediante la herramienta MongoDB Compass. En ella se puede observar la colección de empleados, donde se almacenan los registros con sus respectivos campos como nombre, cargo y salario.

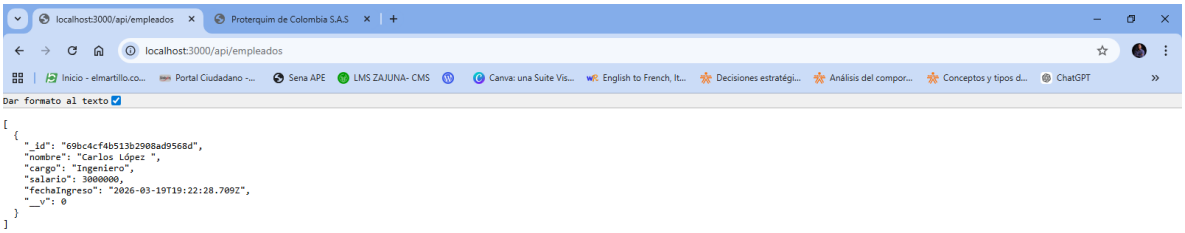
Esta evidencia confirma la correcta persistencia de los datos y la integración entre el backend desarrollado en Node.js y la base de datos.

A continuación, se presenta la base de datos utilizada en el sistema:



Servicio API

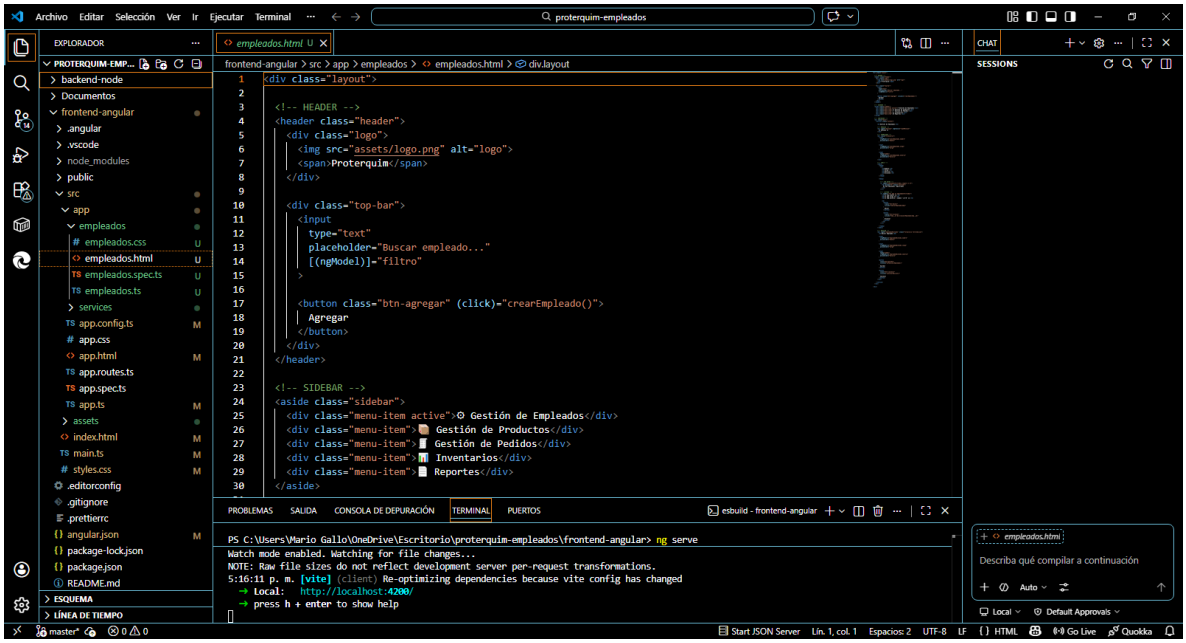
La siguiente imagen muestra la respuesta del servicio API REST al realizar una petición GET al endpoint /api/empleados. Se observa el formato JSON con los datos almacenados en la base de datos, lo cual evidencia el correcto funcionamiento del backend y la disponibilidad de los servicios para ser consumidos por el frontend.



5.14 Entorno de Desarrollo (IDE)

El desarrollo del sistema se realizó utilizando el entorno de desarrollo Visual Studio Code, el cual permitió la edición de código, integración con Git, ejecución de scripts y gestión de dependencias.

Se utilizaron extensiones para mejorar la productividad, como formateadores de código, resaltado de sintaxis y herramientas para Angular y Node.js.



5.15 Control de Versiones

Se utilizó Git como sistema de control de versiones, permitiendo llevar seguimiento de los cambios en el código fuente, facilitar la organización del desarrollo y mantener versiones del proyecto.

Link del repositorio: https://github.com/MG2079/Proterquim-empleados

5.16 Librerías Utilizadas

En el frontend se utilizaron librerías propias de Angular como HttpClient para consumo de APIs y FormsModule para manejo de formularios.

En el backend se utilizó Express para la gestión de rutas y Mongoose para la conexión con MongoDB.

5.17 Ambientes de Desarrollo

El sistema fue desarrollado en un entorno local utilizando Node.js y Angular, y probado mediante ejecución en navegador web.

El backend se ejecutó en el puerto 3000 y el frontend en el puerto 4200.

V. Conclusiones

El desarrollo de este proyecto permitió aplicar conocimientos fundamentales en el área de análisis y desarrollo de software, especialmente en la construcción de aplicaciones web bajo una arquitectura moderna.

Se logró implementar un sistema funcional que cumple con los requerimientos planteados, integrando tecnologías actuales y aplicando buenas prácticas de programación.

Además, se fortalecieron habilidades en el uso de frameworks, consumo de APIs y estructuración de proyectos de software.

El sistema desarrollado puede ser escalado fácilmente para integrar los demás módulos del sistema Proterquim, como inventarios, productos y pedidos.

VI. Bibliografía

- ✓ Angular Documentation. https://angular.io
- ✓ Node.js Documentation. https://nodejs.org
- ✓ Express.js Documentation. https://expressjs.com
- ✓ MongoDB Documentation. https://www.mongodb.com
- ✓ Pressman, R. (2014). *Ingeniería del Software: Un enfoque práctico*.
- ✓ Sommerville, I. (2011). *Ingeniería de Software*